Angelo Miculescu
14 / 07/ 24

AIB Professional Academy Certificate in Database Fundamentals

Database Setup

The database was set up as instructed in the assignment handout. First, SQL code was used to create each individual table, taking care to select appropriate data types for each field (so as to optimize the amount of space taken up) as well as setting up any constraints required (primary / foreign keys etc.). Once this was done, the import function in pgAdmin was used to import each individual csv file into the appropriate SQL relation. Lastly, once all queries were completed, a note was taken of all the fields that were consistently used in join conditions / where statements and these were made into indexes to optimize the queries. This did not include any fields that were already given the title of a primary key as these are already assigned an index in postgreSQL (1).

As for SQL code formatting, an indented style was used.

```
16  CREATE INDEX idx_enroll_dname ON enroll(dname)
```

Data Output | Messages | Notifications

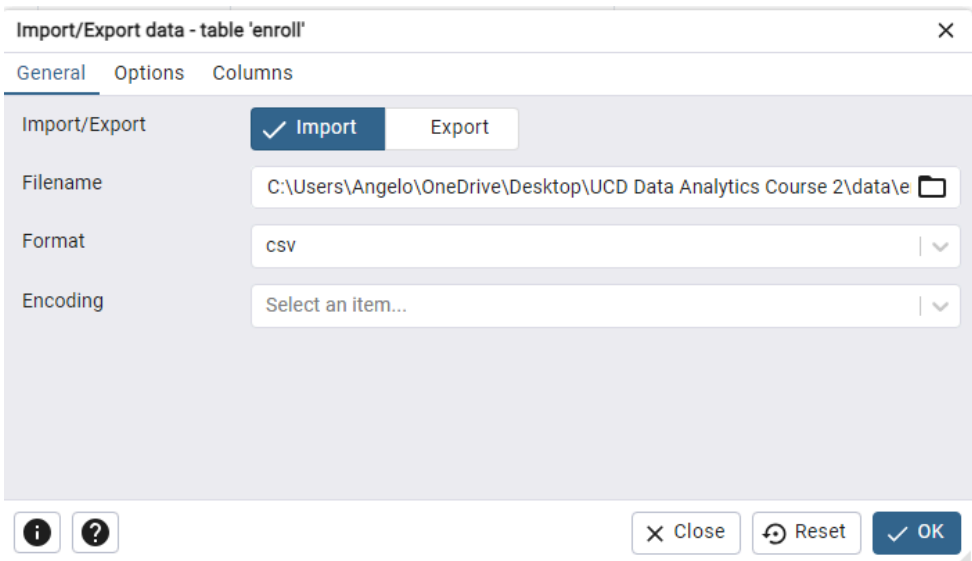
```
CREATE INDEX

Query returned successfully in 56 msec.
```

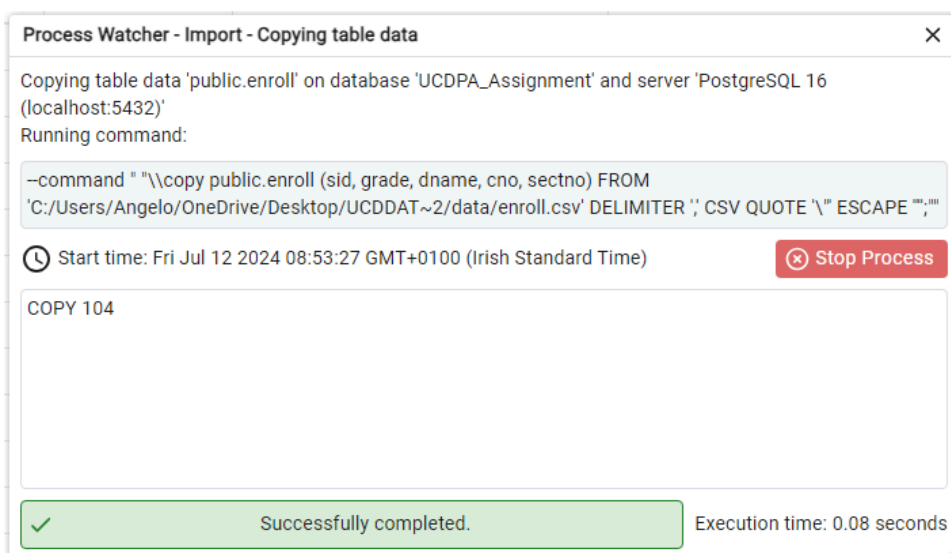
Index creation query and result

The example database I will explain will be the enroll database. For starters, all numbered fields were assigned either INT or SMALLINT, except grade which was given the DECIMAL data type. A composite primary key was used for this relation of sid, cno and sectno. Foreign keys were sid, which was referencing student's sid, and a composite (cno, sectno) referencing that key in section.

```sql
CREATE TABLE enroll (
    sid int NOT NULL,
    grade decimal (2,1) not null,
    dname VARCHAR(23) NOT NULL,
    cno INT NOT NULL,
    sectno SMALLINT NOT NULL,
    PRIMARY KEY (sid, cno, sectno),
    FOREIGN KEY (sid) REFERENCES student(sid),
    FOREIGN KEY (cno, sectno) REFERENCES section(cno, sectno)
    )
```

Once this query was run and the table created, the import function was used to add the data from the csv files into the table:





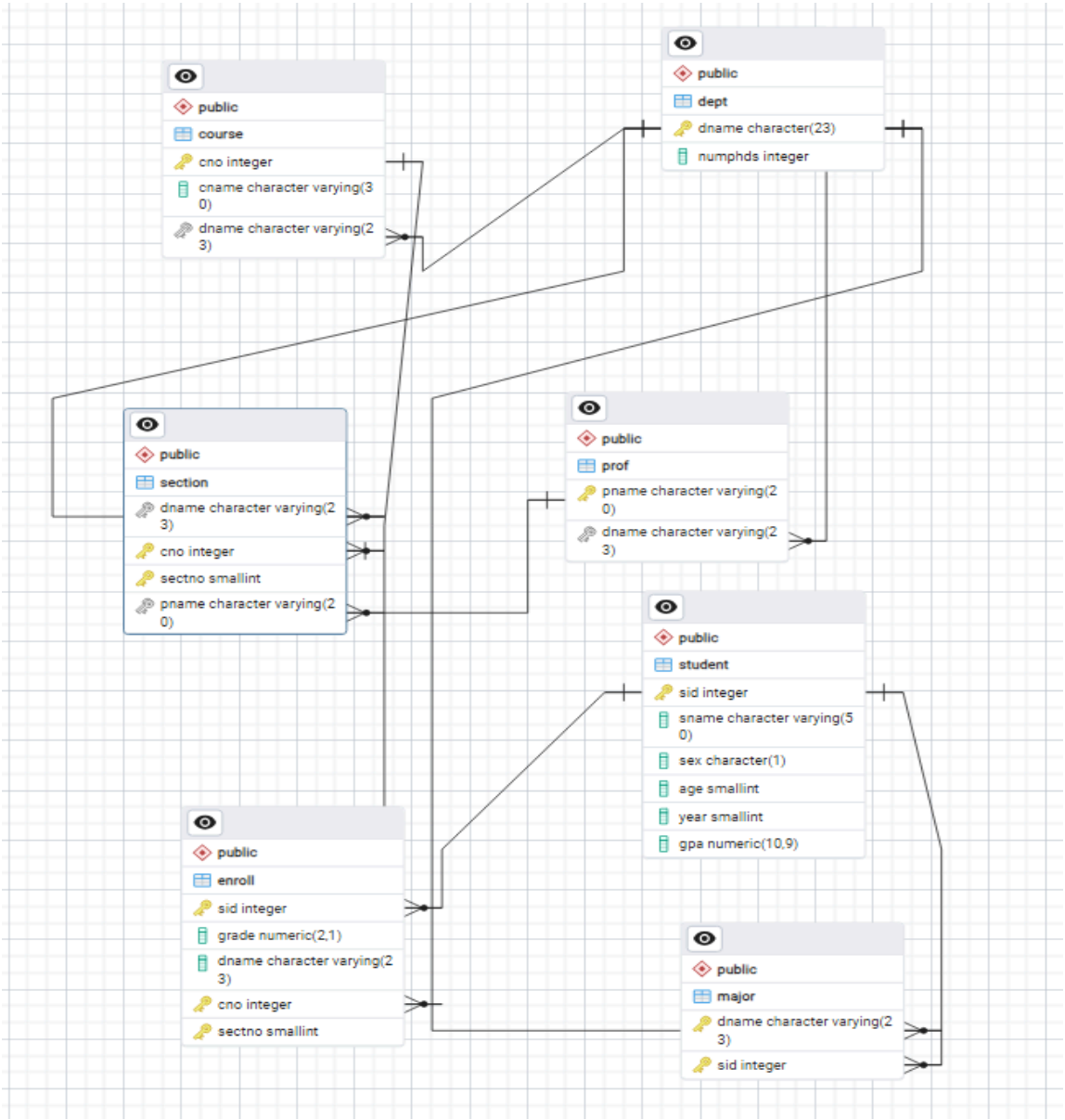
Lastly, the index was created for dname (screenshot above).

To provide a quick overview of the relations and the connections between them:

Table	Primary Key	Foreign Key	Comment
Student	sid	-	One of the main tables you would join to to learn about the students
Dept	dname	-	Join to learn about the number of phd's in each dept. Usually you wouldn't use a character field as a primary key (easy to input wrong figure and create duplicates) but in this case, it was the best choice
Prof	pname	dname -> Dept(dname)	Lists the professors and can join to Dept using dname
Course	cno	dname -> Dept(dname)	Gives the course name and number, can join to Dept, can also join to prof to get a comprehensive list of what professors teach which courses
Major	(dname, sid)	dname -> Dept(dname), sid -> Student(sid)	Requires a composite key as neither dname nor sid are unique in the table.
Section	(cno, sectno)	dname -> Dept(dname), cno -> Course(cno), pname -> Prof(pname)	Once again a composite primary key was required. Can join to a myriad of tables to obtain cname, phd's in the department, professors
Enroll	(sid, cno, sectno)	sid -> Student(sid), (cno, sectno) ->	Required 3 fields to make a primary key,

		Section(cno, sectno)	an important table as it can join onto almost all other tables, sometimes acts like a bridge between tables (eg. Course -> Enroll -> Students)

I also discovered functionality in pgAdmin that creates an ER-Diagram for you so I have obtained a screenshot for that as well:



Queries and Results

Answers to all questions found below. Explanations included if the answer is more tricky.

1. Question 1

A simple where statement was used to single out only the cases with numphds < 50

	pname character varying (20)	numphds integer
1	Jones J.	47
2	Smith S.	41
3	Brian C.	47
4	Edison L.	32
5	Bucket T.	3
6	Walter A.	41

2. Question 2

The list was ordered by gpa and the top 10 were selected using limit 10.

	sname character varying (50)	gpa numeric (10,9)
1	Jetplane Leaving O.	0.000000000
2	Connors Jimmy	0.200000003
3	Ripper Jack T.	0.500000000
4	Smith L.	0.699999988
5	Smith Ike Z.	1.100000024
6	Jones J.	1.100000024
7	Gooch	1.399999976
8	Ziebart F.	1.799999952
9	News Nightly	1.899999976
10	Roger Blotter N.	1.899999976

3. Question 3

Joined course to section to obtain the section. Another two joins were then necessary (to enroll and then student) to obtain gpa. Should be noted that course was only necessary to give the courses their names, otherwise the course number from section would've been enough and it would've been one less join.

	cname character varying (30)	cno integer	sectno smallint	avg numeric
1	Intro to Programming	302	1	2.9400000095000000
2	Intro to Programming	302	2	3.4142856938571429
3	Intro to Data Structures	467	1	3.2600000262000000
4	Compiler Construction	701	1	3.0500000195000000
5	Nonlinear Programming	726	1	2.9999999915000000

4. Question 4

Utilized the having statement when working with aggregated fields (count(st.sid) in this case)

	cname character varying (30)	sectno smallint	no_of_students bigint
1	College Geometry 1	1	7
2	Nonlinear Programming	1	14
3	Compiler Construction	1	10
4	Intro to Data Structures	1	10
5	Intro to Programming	1	10
6	City Planning	1	7
7	Manpower Utilization	1	7
8	Advanced City Planning	1	11
9	Intro to Programming	2	7
10	Highway Engineering	1	9

5. Question 5

	dname [PK] character	no_of_classes bigint
1	Mathematics	2
2	Chemical Engineerin...	1
3	Computer Sciences	4
4	Industrial Engineering	1
5	Civil Engineering	3
6	Sanitary Engineering	2

6. Question 6

	dname character varying (23)	age smallint
1	Industrial Engineering	15
2	Mathematics	16

7. Question 7

This question relied on you knowing that the department in the table major pertained to the department in which the student majored in and not that of the course. Therefore it was necessary to use enroll to find the students studying geometry and then use major to get their actual majors.

	sname character varying (50)	major character varying (23)
1	Sulfate Barry M.	Computer Sciences
2	Cheong R.	Computer Sciences
3	Gooch	Computer Sciences
4	Korpel E.	Computer Sciences
5	Surk K.	Computer Sciences
6	Baker C.	Chemical Engineering
7	Liu Huihusan	Chemical Engineering
8	Trude T.	Civil Engineering
9	Moeri S.	Civil Engineering
10	Roger Blotter N.	Civil Engineering
11	Kool Joseph F.	Civil Engineering
12	Ripper Jack T.	Industrial Engineering
13	Heilskov G.	Industrial Engineering

8. Question 8

Used the query from question 7 as a subquery to get the departments who's majors study geometry and make sure that we only get those that don't contain any of those students

	dname character varying (23)	numphds integer
1	Mathematics	129
2	Sanitary Engineering	3

9. Question 9

	sname character varying (50)
52	Jones Ivan L.
53	Mayer N.
54	Gonring J.
55	Longlastname A.
56	Ghandi I.
57	Uoiea Z.
58	Birch M.
59	Morgan D.
60	Taylor R.
61	Gringo C.
62	Davis Scott P.
63	Bates Michael L.
64	Baskett Wayse T.

10. Question 10

Used simple functions, min and max, to get the appropriate ages and then subtracted them from each other.

	largest_age_difference smallint
1	38

11. Question 11

	dname character varying (23)	avg numeric
1	Civil Engineering	0.00000000000000000000
2	Computer Sciences	0.69999998800000000000
3	Industrial Engineering	0.35000000150000000000

12. Question 12

This query involved finding the students that studied Civil Engineering but singling out those who do all courses in the department (in this case it was all but conditions added in case it wasn't). To do this, I joined student and enroll which would give me 3 entries for the three civil engineering courses, each having a duplicate value for dname. I turned this into a count and selected only the students who had a count that equaled 3.

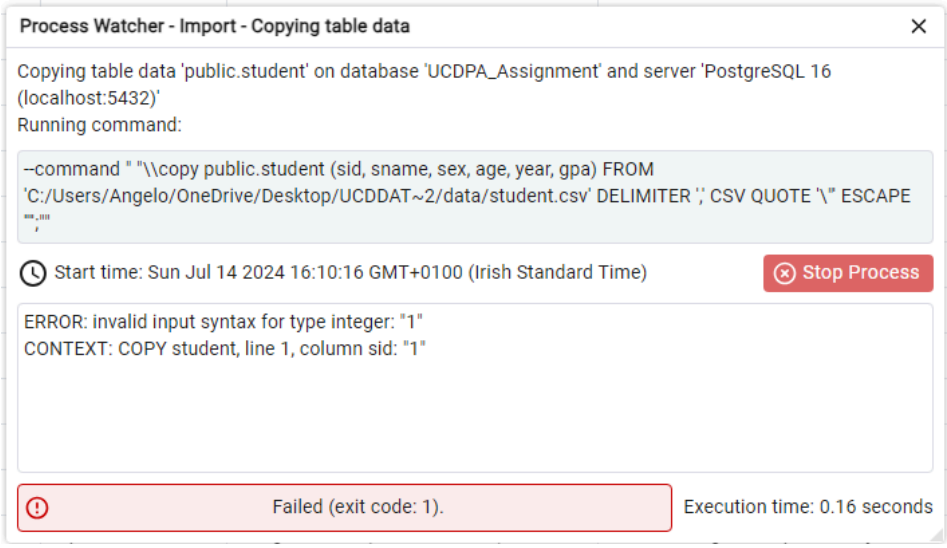
	sid [PK] integer	sname character varying (50)	gpa numeric (10,9)	no_of_courses bigint
15	77	Paull Thomas H.	2.599999905	3
16	54	Maximillian	3.000000000	3
17	88	Bates M.	3.299999952	3
18	34	Kasten Norman L.	2.500000000	3
19	41	Fisher C.	3.500000000	3
20	3	Zeene Ben N.	3.900000095	3
21	70	Caucutt B.	3.000000000	3
22	43	Ksar J.	3.400000095	3
23	90	Zappa F.	4.000000000	3
24	33	Chao Tsechih	3.599999905	3
25	9	Smith Joyce A.	2.000000000	3
26	35	Mathews John W.	3.599999905	3
27	76	Zorhoff C.	3.000000000	3

I should add the query execution plans were analyzed and modified if I felt like there was an improvement to make. Most improvements were seen once indexes were created.

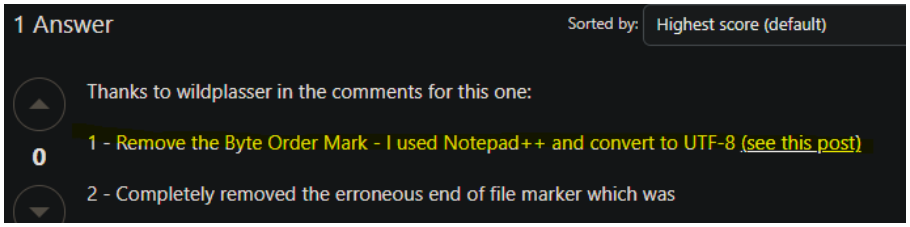
```
Explain Analyze SELECT
        st.sid,
        sname,
        gpa,
        COUNT(e.dname) AS No_Of_Courses
FROM
        student st  --summed up dname so that it gives the number of courses...
    LEFT JOIN
      enroll e                                          --...from each department each stu
        ON st.sid = e.sid
    LEFT JOIN
      course c
        ON e.dname = c.dname
```

Comments & Decisions Made

1. The first decision that required making was changing the encoding on the student csv file. The original was done in UTF 8 - BOM which would leave a single extra byte at the start of the file and give the following error:



- I followed the advice I found below and used my code editor to change the encoding to UTF 8, allowing it to work as intended.



(4)

2. Next change that I was required to make was to the Section relation. The course number for one of the courses showing up in the Sanitary Engineering department was coming in with the same value for another course in the Civil Engineering department which structurally didn't work but also wouldn't make sense unless it was the same course pertaining to two different departments. This value was manually changed from 561 -> 315 to match one of the courses in Sanitary Engineering, Intro to Garbage.

3. Lastly, enroll included 121 students and sid's whereas neither major nor student itself had more than 104. It wouldn't make sense for student to not contain these 17 extra students so it was decided to be a data quality issue, therefore, enroll was cut down also to 104 to accommodate the joins.

References:

1. https://stackoverflow.com/questions/42898988/is-primary-key-automatically-indexed-in-postgresql#:~:text=3%20Answers&text=PostgreSQL%20automatically%20creates%20an%20index,CREATE%20INDEX%20for%20more%20information.) - Used to learn that primary keys are automatically indexed
2. https://www.w3schools.com/sql/sql_check.asp - Learned how to use the Check constraint to make table creation more efficient space-wise

3. https://www.sqlshack.com/understanding-sql-decimal-data-type/ - Data type that I had never used before and needed guidance on
4. https://stackoverflow.com/questions/48253089/copy-from-file-error-invalid-input-syntax-for-integer-%C3%AF-1 - Used to solve issuewith data